RT-KalmanNet

Andrea Marigo, Leonardo Luigi Pepe, Francesca Zorzi

August 2026

1 Running the code

Before running the code, you need to install the required dependencies. You can do this using either `pip` or `conda`. Training and evaluation only require a CPU, no GPU is needed.

Option 1: Using pip

```
pip install -r requirements.txt
```

Option 2: Using conda

If you prefer Conda, you can recreate the exact environment using the provided `environment.yml` file:

```
conda env create -f environment.yml
conda activate rt_kalman_net
```

Once you have finished running the code, you can remove the conda environment to free up space:

```
conda deactivate
conda env remove --name rt_kalman_net
```

Training and evaluation only require a CPU, no GPU is needed.
The code is organized around two Jupyter notebooks, meant to be run top to bottom:

- `CODE/RT_KFNET/task3.ipynb` trains and evaluates the proposed RT-KalmanNet against the original implementation and the REKF on a least-favorable dataset.
- `CODE/RT_KFNET/task4.ipynb` compares all five estimators on the non-linear model.

In both notebooks (`task3.ipynb` and `task4.ipynb`), only the **Configuration** cell (section 2) is meant to be edited: it centralizes the estimators to run, the dataset splits and sizes, the model hyperparameters, and the training settings.
To easily manage the execution flow, the configuration exposes a main `RETRAIN` flag:

- `True`: Trains the models from scratch and saves both the checkpoints and the training/validation history to disk.
- `False` (default): Skips training and directly loads the pre-trained checkpoints and saved history. This ensures that all loss curves, evaluation tables, and plots can be perfectly reproduced without running a new training session. (If a checkpoint or history file is missing, the notebook explicitly halts rather than silently retraining).

Additionally, in `task4.ipynb`, a `REGEN_DATA` flag controls whether to generate new datasets or reuse the cached ones, allowing for fully reproducible experiments.

1.1 How to review the changes

The delivered repository has two commits: the first is the original code as given, the second contains all our modifications applied at once. The `git diff` between them is therefore a complete map of every change and every new file added, serving as the main reference to consult for the detailed modifications.

The original files were never overwritten: our RT-KalmanNet variant lives side by side with the given one, and both are driven by the same pipeline, so their results are directly comparable. Throughout the code and this report, the given implementation is referred to as **original** and ours as **proposed**.

2 Tasks 1 & 2: GRU trained with BPTT

2.1 Gradient flow in the original implementation

The original module is written to be evaluated one step at a time rather than trained through time. Two independent points therefore interrupt the gradient path, each sufficient on its own to leave the weights without gradient:

1. **Feedback detach.** The output feedback is stored as `self.previous_output=final_output.clone().detach()`: it carries the *value* of $\hat{c}_{t-1}$ but not the gradient, so the graph closes at every step and no recurrence survives inside the module.

2. θ **bisection.** θ_t is the root of $\gamma(P_{t+1}, \theta_t) = \hat{c}_t$, found by bisection, where `self.c` enters **only as an `if` condition**; autograd does not differentiate through branches, so the path $\hat{c} \to \theta \to V$ is interrupted as well.

`check_gradient_cut.py` confirms both on a common trajectory: with the original filter the backward pass reaches none of the network parameters, and removing the `detach()` alone does not change this, because the second cut remains. Repairing them is the subject of Sections 2.2 and 2.3.

2.2 Architecture

The interface of the module is unchanged: it maps the filter features $F_1, \dots, F_4$ at time t to the scalar tolerance $\hat{c}_t \in (0, 1)$ consumed by the REKF recursion. Only the internals change.

	Original	**Proposed**
Encoder	`Linear(8→10)` + ReLU	`Linear(8→64)` → LayerNorm → GELU → `Linear(64→64)` → GELU
Body	5 × `Linear(20→20)` + ReLU	1 × `nn.GRU(64, 64)`
Head	`Linear(20→1)` + sigmoid	`Linear(64→1)` + sigmoid
Recurrent state	$\hat{c}_{t-1}$ (scalar), stored in the module	$h_t \in \mathbb{R}^{64}$, explicit argument
Signature	`forward(x)` → `c_t`	`forward(x, h_t)` → `(c_t, h_new)`

Two points matter. The recurrent state is now **returned** by `forward` instead of being kept inside the module, so the caller owns it and can detach it where truncation is wanted. And it is a gated 64-dimensional vector rather than a scalar squashed by a sigmoid: as the first analysis of `check_gradient_cut.py` shows, this is what lets the gradient propagate much further back in time.

2.3 Making $\hat{c}$ differentiable

θ has no closed form in $\hat{c}$: it is defined implicitly as the root of $\gamma(P, \theta) = \hat{c}$. To backpropagate, autograd needs an *expression* linking θ to $\hat{c}$ that is correct both in value and in derivative. We build it with the implicit function theorem: differentiating the identity $\gamma\big(P, \theta(P, \hat{c})\big) = \hat{c}$ gives

$$d\theta \;=\; \frac{d\hat{c} \;-\; \big\langle \partial_P \gamma(P, \theta^\star),\, dP \big\rangle}{\gamma'(P, \theta^\star)},$$

where $\theta^\star$ is the root and γ' the derivative of γ with respect to its second argument. The bisection is left untouched and untracked; afterwards we replace the root by the surrogate

$$\theta \;=\; \theta^\star \;+\; \frac{\hat{c} - \gamma(P, \theta^\star)}{\gamma'(P, \theta^\star)},$$

with $\theta^\star$ and γ' detached, so that only $\hat{c}$ and P carry gradient. The value is unchanged, since the bisection residual $|\gamma - \hat{c}|$ is below its stopping tolerance by construction, while the derivative is exactly the one above.

Keeping $\gamma(P, \theta^\star)$ **attached** is what supplies the $\partial_P \gamma$ term, which autograd then provides without hand-coding its closed form. This matters because P_{t+1} is itself produced by the filter from V_t, hence from $\hat{c}_{t-1}$, $\hat{c}_{t-2}$, and so on: detaching it would cut the dependence of θ_t on the whole past of the recursion. The term is not negligible at the operating point where training starts, and its cost is a small fraction of the forward and backward pass.

The construction is checked in `check_gradient_cut.py`, which compares the resulting $d\theta/d\hat{c}$ against finite differences on the bisection.

2.4 Loss

The loss is the MSE of the **posterior** $\hat{x}_{t|t}$ against the target, i.e. $\|x_t - \hat{x}_{t|t}\|^2$ as in TSP III.D, the section Task 1 refers to. The original script used the one-step-ahead prediction instead, with a slice misaligned by one step. The posterior is also the quantity EKF and KalmanNet output, so Task 4 compares the five estimators on the same object.

Training is mini-batch SGD on a fixed dataset, with the loss averaged over the batch as in TSP III.D; the original script regenerated both training and validation data at every epoch. Note that the parameter `n_steps` counts **epochs** for RT-KalmanNet and gradient steps for KalmanNet.

2.5 BPTT and the truncation variant

With both cuts repaired, BPTT needs no dedicated machinery: the gradient flows back across time steps so a single `loss.backward()` on the trajectory loss is full BPTT.

Following the TSP recommendation, training uses a truncated warm-up over the first fraction of the epochs, then full BPTT. Our truncation, however, is not one of the three variants listed in TSP III.D. Those truncate at the level of the **data**: trajectories are split into short segments treated as independent samples. We truncate at the level of the **graph**: the forward pass runs uninterrupted over the whole trajectory, and every K steps only the gradient path is cut, by detaching h_t and the filter state without resetting them.

3 Task 3: constant and time-varying tolerance

3.1 Data generation

In the minimax robust filter framework, the parameter c represents the **tolerance with respect to the nominal model**, i.e., the mismatch budget allowed in constructing the least-favorable density. It defines the radius of the class of models considered plausible around the nominal density:

$$\mathcal{B}_t = \left\{ \tilde{\phi}_t : D_{\mathrm{KL}}(\tilde{\phi}_t, \bar{\phi}_t) \leq c_t \right\}.$$

Within this class, the robust filter considers the model that maximizes the estimation error, thus obtaining a minimax solution. In other words, it simulates an "adversary" trying to create the worst possible scenario for the filter within this time-varying budget $c(t)$.

In the case of Task 3, the ground-truth tolerance $c_{GT}(t)$ is not just a parameter used by the filter: **it directly enters the generation of the least-favorable model**. The parameter propagates through the following chain:

$$c_{GT}(t) \to \theta_t \to V_{t+1} \to \Phi_t \to (K_t, H_t, L_t) \to (A_e, B_e),$$

which ultimately modifies the probability distribution of the generated trajectories.

Because the actual shape of the targeted disturbance and the noise magnitude derive directly from the budget $c_{GT}(t)$, this parameter is an integral part of the data-generating law. This is why in Task 3 the tolerance leaves an observable statistical footprint in the data and can be identified, unlike in Task 4 where the parameter c does not generate the data, but only exists as a decision variable within the filter.

3.2 Tolerance profiles

The data generation script (`LFD.m`) supports different time-varying profiles for the tolerance parameter $c(t)$, selected via the `c_type` variable. We focus on the two configurations:

1. **Constant:** A baseline time-invariant tolerance, $c(t) = c_{\text{center}}$. For example, this is used in the `data_constant_01.mat` dataset with a fixed $c = 0.1$.

2. **Linear:** A time-varying tolerance defined by a starting value and a constant slope, $c(t) = c_0 + k(t-1)$. The generation script explicitly enforces an admissibility constraint, throwing an error if the trajectory leaves the $[0, 1]$ interval at any point during the T steps. For instance, the dataset `data_linear_001_00001_.mat` starts at 0.01 with a slope of 10^{-4}, meaning c grows from 0.01 to 0.21 over the 2000 simulated steps (reaching 0.11 on the 1000 steps actually used for the forward simulation).

How to Generate New Data: To generate new datasets, the user only needs to edit the configuration variables in `LFD.m` and run the script. The output filename is automatically constructed from the chosen parameters, while the `robust_filter` and `LFM_data` modules handle the rest of the generation pipeline without requiring further modifications.

3.3 Experimental evaluation

The notebook compares three estimators on unseen LFM-generated test trajectories: REKF, the baseline MLP RT-KalmanNet, and the proposed GRU RT-KalmanNet. To ensure unbiased evaluation, models are trained and tested independently for each specific tolerance profile (constant and time-varying).

The experimental protocol assesses performance across four key dimensions:

- **Convergence:** Training and validation losses (in dB) are monitored per epoch to ensure proper optimization and rule out overfitting, which is a critical check for the recurrent model trained via Truncated BPTT on highly correlated data.
- **Computational Efficiency:** Inference time per trajectory is measured to verify if the data-driven single forward pass provides a practical speedup over the analytic REKF, which requires solving a nonlinear equation via bisection at every time step.
- **Filtering Accuracy:** The overall performance is quantified by the Mean Squared Error (MSE) of the state estimates against the ground truth. The dispersion across individual test trajectories is deliberately reported to highlight systematic improvements over the baseline.
- **Tolerance Identification:** The most crucial evaluation compares the network's dynamic tolerance output $\hat{c}(t)$ against the ground-truth budget $c_{GT}(t)$ used by the LFM. Since $c_{GT}(t)$ dictates the generative law of the data, a strong agreement between $\hat{c}(t)$ and $c_{GT}(t)$ proves that the network successfully recovers the underlying model mismatch directly from the observations, rather than merely minimizing the state error by chance.

4 Task 4: comparison on the non-linear model

- Five estimators on the same test trajectories: EKF, REKF, KalmanNet, original RT-KalmanNet, proposed RT-KalmanNet.
- Synthetic non-linear model: we generated trajectories of 100 time steps, building 4 datasets at different measurement-noise levels $1/r^2$ for each of the two scenarios (full and partial information) for a total of 8 datasets.

4.1 Experimental setup

Synthetic Non-Linear Model. The experiment reproduces the benchmark from KalmanNet paper with state and observation dimensions $m = n = 2$. The evolution and observation maps are applied component-wise:

$$f(x) = \alpha \sin(\beta x + \varphi) + \delta, \qquad h(x) = x^2$$

The process and measurement gaussian noise covariances are defined as $Q = q^2 I_2$ and $R = r^2 I_2$.

Information Scenarios. To evaluate robustness against model mismatch, the filters operate under two distinct scenarios:

- **Full information:** The filters are provided with the exact true dynamics ($\alpha = 0.9$, $\beta = 1.1$, $\varphi = 0.1\pi$, $\delta = 0.01$).
- **Partial information:** The filters are provided with a mismatched, simplified model ($\alpha = 1$, $\beta = 1$, $\varphi = 0$, $\delta = 0$).

Crucially, the underlying data are always generated using the *true* parameters. The partial setting only affects the evolution model supplied to the filters.

Noise Parameterization and Controlled Comparison. The evaluation across four measurement-noise levels, indexed by $1/r^2 \in \{-12.04, -6.02, 0.0, +10.0\}$ dB. The process-to-measurement noise ratio is kept constant at $q^2/r^2 = -20$ dB, ensuring the two noise powers scale together. For values of $1/r^2 > +10$ dB we did not observe any significative change in filters performances.

4.2 Results: Mean Squared Error (dB) & Inference Time

Note: MSE values are in dB. Inference time is measured in ms/step at 0 dB over 10 test sequences. (F) = Full information, (P) = Partial information.

Summary of Findings. Under full information, the standard EKF is already near-optimal, and robustification provides no measurable benefit. Under partial information (model mismatch), the proposed recurrent

Estimator	F: -12.04	F: -6.02	F: 0	F: +10	P: -12.04	P: -6.02	P: 0	P: +10	Time (F)	Time (P)
EKF	-6.07	-13.27	**-19.55**	**-29.63**	-2.19	-5.75	-8.29	-8.67	0.31	0.26
REKF	-6.05	-13.26	-19.54	-29.62	-1.85	-4.13	-3.90	**-14.48**	1.05	1.09
KalmanNet	**-6.34**	**-13.30**	-19.52	-29.60	**-6.32**	**-12.55**	**-16.57**	**-19.04**	0.32	0.25
Original RT-KN	-5.64	-13.13	-19.40	-29.47	+2.39	-0.14	-2.73	-13.69	2.11	1.35
Proposed RT-KN	-6.06	-13.26	-19.53	-29.52	-1.59	-4.75	-5.66	-14.47	1.50	1.28

RT-KalmanNet successfully adapts its tolerance dynamically, fixing the inert behavior of the original MLP variant and effectively matching the analytic REKF. However, the purely data-driven KalmanNet ultimately outperforms all tolerance-based filters by a wide margin, achieving the lowest MSE at a fraction of the computational cost.

5 Development, Considerations and Future Work

This section summarises how the project evolved, from the first prototype to the final delivered notebooks.

Development phases

1. **First implementation and validation:** Initial RT-KalmanNet covering both tasks. We diagnosed the `c_t` collapse (weight decay, bias initialisation, feature scaling), fixed the EKF Jacobian, and validated the least-favourable-model filter against the MATLAB and EKF references.

2. **Project restructuring:** Restart from the professor's original code base, reorganised into a clean structure: separate model variants (original and proposed), one subfolder per task, a reusable training-pipeline class, and a reproducible environment.

3. **Task 4:** Implementation and evaluation of the Task-4 pipeline: large-scale training runs, aggregate comparison of the five estimators, a truncated-BPTT warm-up schedule, and an attempt at vectorising the filter for speed.

4. **Task 3:** Implementation of the Task-3 pipeline on the refactored training procedure, with new MATLAB datasets with parameterised time-varying tolerance, dedicated tuning notebooks, and systematic sweeps of learning rate, weight decay, BPTT truncation, gradient clipping and hidden size.

5. **Final tuning and refinement:** Final notebooks for Tasks 3 and 4, with a flexible execution pipeline (train from scratch or load pre-trained checkpoints and histories), concise explanations, and targeted plots to help interpret the results.

Future work

The main direction we identified is **stabilising the training pipeline**, so as to reduce the strong hyperparameter sensitivity observed in Task 3. Alternative regularisation, different loss formulations, or more robust initialisation could recover the smooth $c(t)$ tracking seen in earlier iterations without compromising the mathematical integrity of the filter.